

DevOps & CI/CD

Java Backend Interview — Short Notes

1. DevOps Fundamentals

- **DevOps**: culture + practices + tools that unify software development (Dev) and IT operations (Ops).
- Goal: shorter development cycles, frequent releases, higher quality, faster feedback loops.
- **Key principles**: automation, continuous feedback, collaboration, shared responsibility, fail fast.

Concept	Description	Tools
CI — Continuous Integration	Developers integrate code frequently (multiple times/day). Each push triggers automated build + tests.	Jenkins, GitHub Actions, GitLab CI, Bitbucket Pipelines, CircleCI
CD — Continuous Delivery	Every passing build is potentially deployable. Deployment to production is a manual click.	Argo CD, Spinnaker, AWS CodeDeploy, Octopus Deploy
CD — Continuous Deployment	Every passing build automatically deployed to production. No manual gate.	Same tools. Requires strong test coverage and monitoring.
IaC — Infrastructure as Code	Provision and manage infrastructure using code. Versioned, repeatable.	Terraform, Pulumi, AWS CloudFormation, AWS CDK
Configuration Management	Automate server configuration and software installation.	Ansible, Chef, Puppet, SaltStack
Container Orchestration	Deploy and manage containers at scale.	Kubernetes, Docker Swarm, Amazon ECS
Observability	Logs, metrics, traces — understand system behavior in production.	Prometheus, Grafana, ELK Stack, Jaeger, Datadog
Secret Management	Securely store and inject credentials, API keys, certificates.	HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets

2. CI Pipeline — Best Practices

2.1 Typical CI Pipeline Stages

Stage	What Happens	Fails On	Tools
Checkout	Clone repo at commit SHA.	Missing access, invalid ref.	Git
Build	Compile source code. Catch compile errors.	Compile errors, missing deps.	mvn compile / gradle compileJava
Unit Tests	Run fast unit tests. No external dependencies.	Test failures. Coverage below threshold.	JUnit 5, Surefire, JaCoCo
Code Quality	Static analysis: code smells, bugs, security vulnerabilities.	Quality gate failure.	SonarQube, Checkstyle, SpotBugs, PMD
Integration Tests	Test against real DB/cache/queues via Testcontainers.	Integration failures.	JUnit 5 + Testcontainers, Failsafe
Build Image	Build Docker image. Tag with commit SHA.	Dockerfile errors, push failure.	Docker, Buildah, Kaniko

Security Scan	Scan image for CVEs. SAST/DAST.	Critical/High vulnerabilities.	Trivy, Snyk, OWASP Dependency Check
Push Artifact	Push image to registry. Publish JAR to Nexus/Artifactory.	Registry push failure.	Docker Hub, ECR, GCR, JFrog
Deploy to Staging	Deploy to non-prod environment. Run smoke tests.	Deploy failure, smoke test failure.	Helm, kubectl, Argo CD

2.2 GitHub Actions — Complete Spring Boot Pipeline

```

name: CI/CD Pipeline
on:
  push:
  branches: [main]
  pull_request:
  branches: [main]
jobs:
  build-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env: { POSTGRES_DB: testdb, POSTGRES_PASSWORD: test }
        options: --health-cmd pg_isready
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
      with: { java-version: '21', distribution: 'temurin' }
      - uses: actions/cache@v3
      with: { path: ~/.m2, key: maven-${{ hashFiles('pom.xml') }} }
      - run: mvn clean verify
      - uses: actions/upload-artifact@v4
      with: { name: jar, path: target/*.jar }
  docker-push:
    needs: build-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
      with: { registry: ghcr.io, username: ${{github.actor}},
        password: ${{secrets.GITHUB_TOKEN}} }
      - uses: docker/build-push-action@v5
      with:
        push: true
        tags: ghcr.io/org/app:${{github.sha}}
  deploy:
    needs: docker-push
    environment: production
    runs-on: ubuntu-latest
    steps:
      - run: |
        kubectl set image deployment/app \

```

```
app=ghcr.io/org/app:${{github.sha}}
kubectl rollout status deployment/app
```

3. Deployment Strategies

Strategy	How It Works	Downtime	Rollback	Use When
Recreate	Terminate all old instances, deploy all new.	Yes — brief outage.	Redeploy old version.	Dev/test environments. Acceptable downtime. Stateful apps needing full restart.
Rolling Update	Replace instances one by one (or N%). New and old run simultaneously.	No.	Roll back to previous image.	Default Kubernetes strategy. Stateless services. Easy setup.
Blue/Green	Two identical environments (Blue=live, Green=new). Switch traffic at LB after validation.	No.	Instant — switch back to Blue.	High-risk releases. Need instant rollback. DB migration-sensitive deployments.
Canary	Route small % of traffic (1-5%) to new version. Gradually increase if healthy.	No.	Redirect canary traffic back.	New features with uncertainty. A/B testing. Gradual confidence building.
Shadow	Mirror production traffic to new version. Responses discarded. Compare behaviour.	No.	No traffic impact.	Test new version under real load without user impact.
Feature Flags	Deploy code disabled. Enable for specific users/% via flag config.	No.	Disable flag instantly.	Decouple deployment from release. Dark launches. Kill switch for features.

4. GitOps & Argo CD

- **GitOps:** Git as single source of truth for infrastructure and deployment config. Desired state in Git → automatically applied to cluster.
- **Argo CD:** declarative GitOps CD tool for Kubernetes. Watches Git repo, syncs cluster state to match.
- **Pull model:** agent in cluster pulls desired state from Git (vs push model — CI pushes to cluster).
- **Benefits:** full audit trail (git history = deployment history), easy rollback (git revert), PR-based deployment workflow.

```
# Argo CD Application manifest
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: order-service
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/k8s-configs
    targetRevision: main
    path: apps/order-service/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
```

```
prune: true # delete resources removed from git
selfHeal: true # revert manual cluster changes
```

5. Jenkins

5.1 Jenkins Concepts

Concept	Description
Master/Controller	Jenkins server — schedules jobs, manages agents, stores configs and build history.
Agent/Node	Machine that runs build jobs. Can be static VMs or dynamic (Kubernetes pods).
Job/Project	Freestyle Job (UI config), Pipeline Job (Jenkinsfile), Multibranch Pipeline (per-branch).
Jenkinsfile	Pipeline as code in Groovy DSL. Checked into repo alongside source code.
Stage	Logical phase of pipeline (Build, Test, Deploy). Shown in Stage View UI.
Step	Single action within a stage (sh, bat, echo, withCredentials).
Plugin	Jenkins extended via 1800+ plugins. Maven, Docker, Kubernetes, SonarQube, Slack, etc.

5.2 Declarative Jenkinsfile

```
pipeline {
  agent { kubernetes { yaml """
spec:
containers:
- name: maven
image: maven:3.9-eclipse-temurin-21
""" } }

  environment {
    IMAGE = "myregistry/order-service:${GIT_COMMIT}"
    DOCKER_CREDS = credentials('docker-hub-creds')
  }

  stages {
    stage('Build & Test') {
      steps { sh 'mvn clean verify' }
      post { always { junit 'target/surefire-reports/*.xml' } }
    }
    stage('Docker Build & Push') {
      steps {
        sh "docker build -t ${IMAGE} ."
        sh "echo ${DOCKER_CREDS_PSW} | docker login -u ${DOCKER_CREDS_USR} --password-stdin"
        sh "docker push ${IMAGE}"
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps { sh "kubectl set image deployment/order-service app=${IMAGE}" }
    }
  }
  post {
    failure { slackSend channel:'#alerts', message:"FAILED: ${env.JOB_NAME}" }
  }
}
```

6. DORA Metrics — Measuring DevOps Performance

Metric	What It Measures	Elite Performance	How to Improve
Deployment Frequency	How often code is deployed to production.	Multiple times per day.	CI/CD automation, trunk-based dev, small PRs.
Lead Time for Changes	Time from commit to production.	Less than 1 hour.	Faster CI, parallel pipelines, automated testing.
Change Failure Rate	% of deployments causing incidents.	Less than 5%.	Better testing, canary deployments, feature flags.
Time to Restore	Time to recover from production incident.	Less than 1 hour.	Runbooks, on-call, rollback automation, chaos engineering.

7. Interview Questions

CI/CD Fundamentals

Q1. What is the difference between Continuous Delivery and Continuous Deployment?

→ Continuous Delivery: every passing build is in a deployable state. Production deployment is a deliberate manual trigger. Continuous Deployment: every passing build is automatically deployed to production — no human gate. CD requires strong test coverage and monitoring. Many companies use Delivery (not Deployment) for safety.

Q2. How do you design a CI/CD pipeline for a Spring Boot microservice?

→ (1) Trigger on push/PR. (2) Compile + unit tests + coverage gate. (3) Code quality scan (SonarQube). (4) Integration tests with Testcontainers. (5) Build Docker image tagged with commit SHA. (6) Security scan (Trivy). (7) Push to registry. (8) Deploy to staging. (9) Smoke tests. (10) Manual approval → deploy to production. (11) Monitor rollout.

Q3. What is GitOps? How does Argo CD implement it?

→ GitOps: Git is single source of truth for deployment config. All changes via PR. Argo CD: controller running in Kubernetes watches Git repo with Helm charts / kustomize manifests. Continuously compares desired state (Git) with actual state (cluster). Syncs automatically on divergence. selfHeal=true reverts manual kubectl changes. Full audit trail in git.

Deployment & Operations

Q4. Blue/Green vs Canary deployment — when to choose each?

→ Blue/Green: maintain two identical environments. Switch all traffic instantly at load balancer. Best for: instant rollback needed, high-risk releases, DB schema changes. Expensive (double infrastructure). Canary: gradual traffic shift (1% → 10% → 100%). Best for: validating with real traffic, A/B testing, gradually building confidence. More complex routing but lower cost than blue/green.

Q5. What happens if a deployment goes wrong in production?

→ (1) Automated rollback: kubectl rollout undo or Argo CD sync to previous Git commit. (2) Feature flag kill switch — disable the feature instantly. (3) For blue/green: switch LB back to blue immediately. (4) For canary: route all traffic back to stable version. (5) Post-mortem: find root cause, fix, add tests to prevent regression. DORA metric: Time to Restore.

Q6. How do you manage secrets in a CI/CD pipeline?

→ Never commit secrets to Git. Options: (1) CI/CD platform secrets (GitHub Secrets, Bitbucket Repository Variables) — injected as env vars. (2) HashiCorp Vault — dynamic secrets with short TTL. (3) AWS Secrets Manager / Azure Key Vault — cloud-native. (4) Kubernetes Secrets (base64 — weak alone) + external-secrets-operator to sync from Vault/AWS. Rotate regularly. Audit access.

Q7. What are DORA metrics? Why do they matter?

→ DORA (DevOps Research and Assessment): 4 key metrics — Deployment Frequency, Lead Time for Changes, Change Failure Rate, Time to Restore. Proven correlation between high DORA performance and business outcomes (revenue, profit, market share). Elite performers deploy multiple times/day with <1hr lead time, <5% change failure rate, <1hr recovery. Used to benchmark DevOps maturity.

